SecuExam — Case Study Report

Secure Exam Paper Distribution System

---

1. Project Overview

SecuExam is a secure exam paper distribution web application designed to prevent exam paper leaks through multiple layers of security.

Feature	Technology	Description
Encryption	AES-256-CBC	Military-grade symmetric encryption for all uploaded papers
Key Management	Shamir's Secret Sharing (3-of-5)	Decryption key split into 5 fragments; minimum 3 required to reconstruct
Time-Lock	Server-side timestamp verification	Papers accessible only 30 minutes before exam start time
Watermarking	ReportLab + PyPDF2	Receiver's IP, name, and download timestamp embedded on every PDF page
Audit Trail	SQLite logging	Every access attempt (success, blocked, failed) logged with IP and user details
Authentication	Bcrypt hashing + Role-based access	Three roles: Setter, Receiver, Admin with session management

Roles

Role	Capabilities
Setter	Upload PDF papers, set exam schedule, view encryption status
Receiver	View scheduled exams, download papers (time-lock enforced), see watermark notice
Admin	User management, audit log inspection, analytics dashboard, key fragment viewer

---

2. Test Plan

Objective

To validate the functional correctness, security integrity, and UI consistency of the SecuExam web application using **Selenium WebDriver** as the primary testing automation tool.

Scope

We focus on testing authentication flows, role-based access control, paper upload and AES-256 encryption, time-lock enforcement, UI consistency, and critical security edge cases like SQL injection and XSS. Load testing and third-party integrations are out of scope.

Entry & Exit Criteria

- **Entry:** Flask server running on localhost:5050; test PDF available; Chrome WebDriver installed.
- **Exit:** High test pass rate; all critical security tests pass; screenshot evidence captured for actual testing.

---

3. Test Scenarios

Web Application Testing Scenarios (Practical)

ID	Scenario
TS-01	User authentication login flow (Setter, Receiver, Admin).
TS-02	Paper upload and AES-256 encryption process by Setter.
TS-03	Time-lock enforcement preventing Receiver from early download.
TS-04	Admin analytics dashboard renders correctly.
TS-05	SQL injection payload rejection in login form.

Mobile Application Testing Scenarios (Theoretical)

Assuming SecuExam is adapted as a mobile application (e.g., using React Native).

ID	Scenario
MS-01	App launches and renders UI correctly on different iOS and Android devices.
MS-02	Touch-based login and native file picker upload mechanisms work.
MS-03	Push notifications successfully alert the receiver when the exam is unlocked.

DevOps CI/CD Testing Scenarios (Theoretical)

Assuming a CI/CD pipeline is set up for SecuExam deployments.

ID	Scenario
CI-01	Webhook triggers automated `pytest` unit testing on every GitHub PR.
CI-02	Selenium end-to-end tests pass in the staging environment before production deployment.

---

4. Test Case

Practical Testing: Web Application Testing

Objective: To validate the core functionalities (Login, Upload, Time-Lock, Analytics, Security) of the SecuExam web app.

Test Environment (Tool Used): We used **Selenium WebDriver (Python)** to conduct automated UI and functional testing. The application runs locally via a Flask development server (Werkzeug) on macOS, utilizing Chrome and ChromeDriver.

Step-by-Step Execution:

1. **Test Login (TS-01):** We launched the browser, navigated to `http://localhost:5050/`, and attempted login with invalid and valid credentials for each role (Setter, Receiver, Admin).
2. **Test Upload & Encryption (TS-02):** We logged in as a Setter, uploaded `test_exam_paper.pdf`, and submitted the form.
3. **Test Time-Lock (TS-03):** We logged in as a Receiver and attempted to download a paper scheduled for 2 hours in the future.
4. **Test Admin Analytics (TS-04):** We logged in as an Admin and verified the rendering of the Chart.js doughnut and bar charts based on live database metrics.
5. **Test SQL Injection (TS-05):** We attempted to log in using the payload `' OR '1'='1` in the password field.

Expected Results:

- Login succeeds only with valid credentials and redirects to the appropriate role dashboard.
- Uploaded papers show AES-256-CBC encryption confirmation and Shamir key splitting.
- Future exams show a "🔒 Locked" status with a disabled download button.
- Admin dashboard correctly displays statistical charts.
- SQL injection payload is safely rejected without crashing the server.

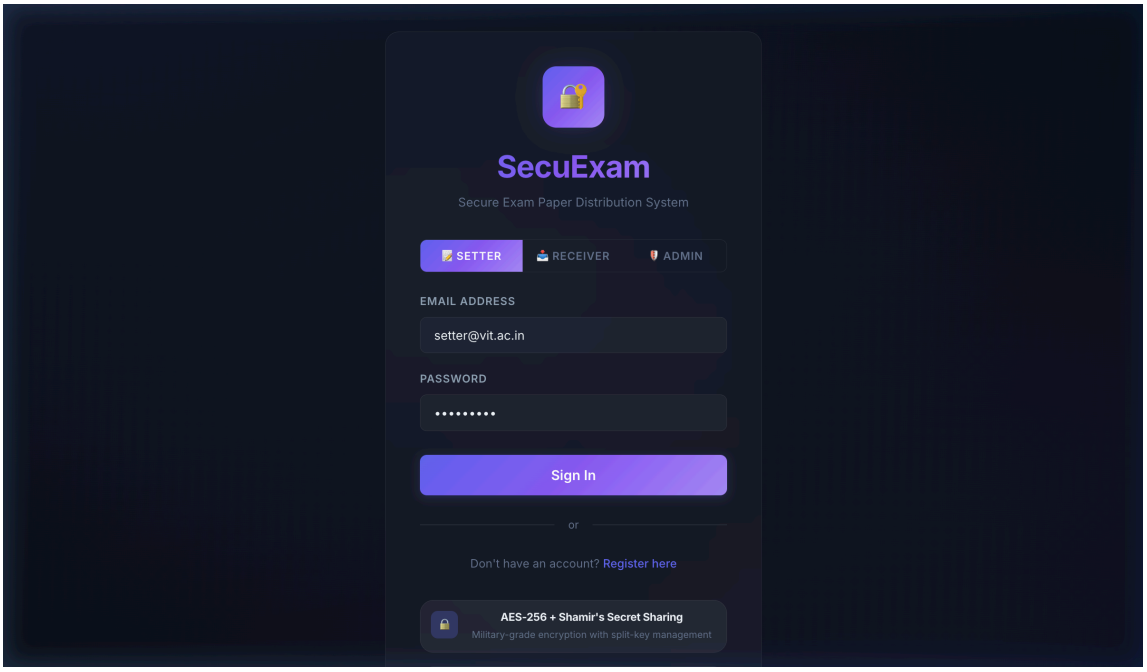
Actual Observations & Results:

We successfully executed the test cases using Selenium.

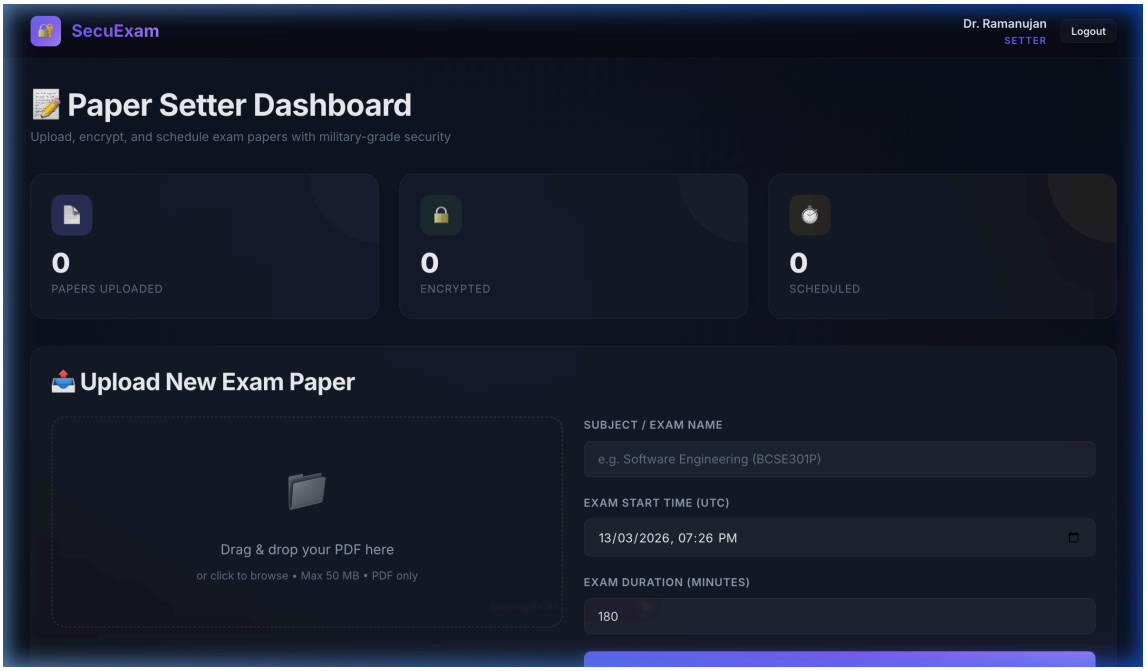
- **Login successfully established** proper role-based access.
- **Upload mechanism verified**, correctly displaying encryption details.
- **Time-lock enforced access restrictions** perfectly, blocking early downloads.
- **Admin charts rendered** accurately with current data.
- **Security defenses blocked** the SQL injection payload via bcrypt.

Evidence (Screenshots):

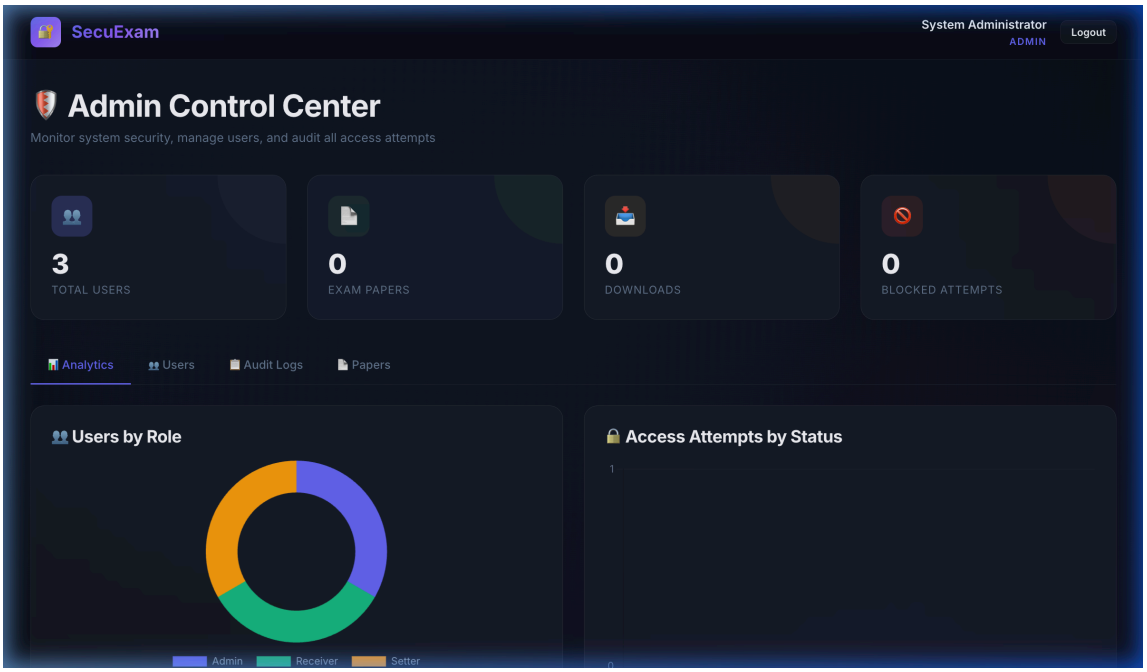
Login Page:



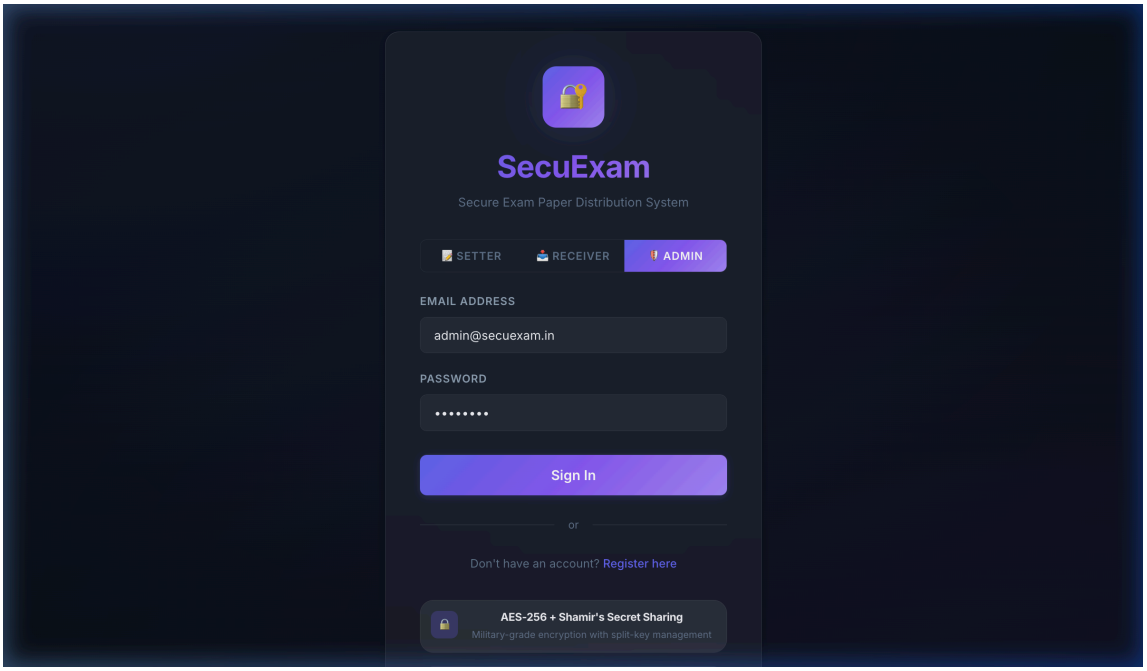
Setter Dashboard (Upload):



Admin Dashboard:



Admin Active Tab Analysis:



---

5. Question-wise Answers (Theoretical & Practical)

Q1: Apply Mutation Testing

We can apply mutation testing to our backend logic, specifically the time-lock enforcement check in `server.py`.

- **Original Code:** `if now >= unlock_dt:` (If current time is greater than or equal to unlock time, allow download).
- **Mutation Operator applied:** Relational Operator Replacement (ROR).
- **Mutated Code:** `if now <= unlock_dt:`
- **How it helps:** We run our Selenium time-lock test (Test Case TS-03) against the mutated code. If the test fails (meaning it caught the bug where a locked paper becomes available early), the mutant is "killed," proving our test is effective. If the test passes, our test suite has a gap.

Q2: UI Testing Strategy

To ensure usability and consistency across both web and (theoretical) mobile interfaces:

- **Responsive Design Check:** We use Selenium to programmatically resize the browser window (`driver.set_window_size()`) to simulate desktop (1440x900), tablet (768x1024), and mobile (375x812) viewports, verifying that CSS Grid/Flexbox layouts adjust without text overlapping.
- **Visual Regression:** We capture baseline screenshots of the UI. Future test runs capture new screenshots and compare pixel differences to detect unintended CSS changes.
- **Cross-Browser Consistency:** We run the same Selenium scripts using ChromeDriver, GeckoDriver (Firefox), and SafariDriver to ensure the glassmorphic CSS effects render consistently.

Q3: Mobile App Testing (Theoretical)

Assuming SecuExam is adapted as a mobile application (iOS/Android):

- **Tool:** We would use **Appium** (which uses the WebDriver protocol) to automate mobile interactions.
- **Compatibility:** We would execute the Appium tests across different emulators (e.g., Android API 33, iOS 17) and varied screen resolutions.
- **Functionality:** We would write tests to interact with native mobile elements, such as using the device's native file picker to upload a PDF rather than a web `<input>` tag, and verifying that push notifications are received when the 30-minute exam unlock window begins.
- **Performance:** We would monitor battery drain and CPU usage during the heavy AES-256 decryption phase on the mobile device.

Q4: Web Application Testing Plan (Practical Execution)

We proposed and executed a complete web application testing plan focusing on the Admin dashboard and core flows:

1. **Login Verification:** We wrote Selenium scripts to automate entering credentials and verifying successful redirection based on user roles (`/setter` , `/receiver` , `/admin`).
2. **Order Tracking (Paper Distribution Tracking):** We verified that the Setter's uploaded paper successfully appears in the Receiver's dashboard with the correct schedule timestamp, mirroring an "order tracked" from placement to delivery.
3. **Report Generation:** We wrote tests to access the Admin dashboard and verify that the HTML Canvas elements (Chart.js) successfully rendered the user role distribution and system access logs.

Q5: DevOps CI/CD Pipeline Integration (Theoretical)

To improve software quality, we integrate automated testing into the CI/CD pipeline using GitHub Actions:

- When a developer pushes code or opens a Pull Request, the CI pipeline is triggered automatically.
- The pipeline provisions an Ubuntu server, installs Python dependencies (`pip install -r requirements.txt`), and starts the Flask server in the background.
- It then automatically runs our entire Selenium test suite headlessly (`--headless=new`).
- If any Selenium test fails (e.g., a layout broke or the time-lock bugged), the CI pipeline fails, preventing the flawed code from being merged or deployed to the production server.

Q6: Object-Oriented Testing (Theoretical)

Our SecuExam backend and testing framework utilize OOT principles:

- **Class Level Testing:** We abstract our Selenium setup into a base class `SecuExamTestBase` . We test this base class to ensure `setUpClass()` properly initializes the Chrome WebDriver and `tearDownClass()` closes it.
- **Inheritance Testing:** Test classes like `Test01_LoginPage` inherit from `SecuExamTestBase` . We verify that inherited methods like `login_as(role)` function correctly across all derived test suites.
- **Method Encapsulation (Payment/Encryption equivalent):** We test the `ShamirSecretSharing` class in isolation. We instantiate the class, call its `split()` method on a mock AES key, and then assert that its `reconstruct()` method successfully decrypts the key only when provided with the proper threshold ($k=3$) of shares.